

void_v1: A Differential-Attention Sub-1B Language Model Specialized for Code and Command-Line Reasoning

Design Paper (v0.1, pre-training)

Krish Kumar Sharma

September 2026

Abstract

We present the design of **void_v1**, a 480M-parameter decoder-only language model targeted at code generation and command-line reasoning, to be trained from scratch on a curated 100B-token corpus using free-tier and low-cost cloud compute. The architecture combines a deep-and-narrow scaling regime with grouped-query attention, RoPE positional encoding with an extended frequency base, a mixed sliding-window / full attention pattern, register tokens, and Multi-Token Prediction heads for training-time auxiliary loss. Our central architectural bet is the adoption of *Differential Attention* at the sub-1B parameter scale, a mechanism previously proposed by [Ye et al. \(2024\)](#) for reducing attention noise, but not yet published in any released model at this size class. We formalise the noise-cancellation argument as an inequality on expected off-diagonal attention mass, derive closed-form parameter and FLOP budgets for the architecture, and compare void_v1 theoretically against five reference models in the 350M–1.3B range (Pythia-410M, MobileLLM-350M, SmolLM2-360M, Qwen2.5-Coder-0.5B, DeepSeek-Coder-1.3B). We further describe a three-stage curriculum with teacher distillation from Qwen2.5-Coder-7B and an accompanying release plan including a novel command-line reasoning benchmark, **void-cli-bench**. This document is a design paper: no empirical results are yet available. All predicted numbers are labelled as such and are conditioned on the training recipe stated herein.

Contents

1	Introduction	2
2	Related Work	3
2.1	Sub-1B language models	3
2.2	Code-specialized small models	3
2.3	Attention mechanism variants	3
2.4	Training-recipe innovations	4
3	void_v1 Architecture	4
3.1	Design principles	4
3.2	Overall configuration	4
3.3	Differential Grouped-Query Attention	5
3.4	Mixed attention pattern	6
3.5	Multi-Token Prediction (MTP) heads	6
3.6	Register tokens	6
3.7	Optimizer choice	7

4	Theoretical Analysis	7
4.1	Attention noise decomposition	7
4.2	Differential attention noise reduction	7
4.3	Parameter efficiency of GQA	8
4.4	MTP as implicit regularization	8
4.5	FLOPs and memory budget	9
5	Training Methodology	9
5.1	Data pipeline	9
5.2	Curriculum: three-stage annealing	10
5.3	Knowledge distillation	10
5.4	Post-training	10
6	Theoretical Comparison with Reference Models	10
6.1	How expected values are derived	11
7	Risks and Ablation Plan	11
8	Evaluation	12
8.1	Standard benchmarks	12
8.2	void-cli-bench	12
9	Conclusion	12

1 Introduction

The sub-1B parameter regime for large language models (LLMs) has, in the last twelve months, become surprisingly competitive. Qwen2.5-Coder-0.5B (Hui et al., 2024) reaches HumanEval pass@1 of 40.5, exceeding several 7B-class general-purpose models on code, while SmolLM2-1.7B (Allal et al., 2025) matches or beats larger models on common-sense reasoning through aggressive over-training and data curation. These results demonstrate that at small scale, *training recipe* — not raw parameter count — is the dominant driver of downstream quality.

Yet the architectures used by these models are strikingly similar: a Llama-family decoder (Touvron et al., 2023) with grouped-query attention (GQA) (Ainslie et al., 2023), rotary positional embeddings (Su et al., 2024), RMSNorm (Zhang & Sennrich, 2019), and a SwiGLU feed-forward network (Shazeer, 2020). Recent research has proposed a range of drop-in architectural improvements (Differential Attention (Ye et al., 2024), Multi-Token Prediction (Gloeckle et al., 2024; DeepSeek-AI, 2024), register tokens (Darcet et al., 2024), QK-normalization (Chameleon Team, 2024; Dehghani et al., 2023), the Muon optimizer (Jordan, 2024)), but no released model below 1B parameters currently combines these techniques.

We propose **void_v1** to fill that gap and, in the process, plant a falsifiable research flag: *Does Differential Attention improve code generation quality at 480M parameters when trained on a code-specialized corpus with distillation from a code-specialist teacher, relative to an otherwise identical standard GQA baseline?*

Contributions. This design paper contributes:

1. A concrete 480M-parameter decoder architecture (§3) combining seven proven-in-2024 techniques at a scale where the specific combination has not been published, with parameter and FLOP budgets derived from first principles (§4.5).

2. A theoretical analysis of Differential Attention as a noise-cancellation operator, formalised as Theorem 1 on the expected off-diagonal attention mass (§4).
3. A theoretical comparison with five reference models in the 350M–1.3B range under identical assumptions (§6).
4. A three-stage training curriculum with teacher distillation and an accompanying command-line reasoning benchmark (§5, §8).
5. An explicit risk register and ablation plan (§7) with graceful-degradation contracts for each novel component.

Scope and limits. This is a *design paper*. No pretraining run has yet been executed. Every quantitative statement about `void_v1` in this document is either (a) a mathematical derivation from the stated configuration, or (b) an *expected value* conditioned on the training recipe and derived by comparison with published models of similar scale. We explicitly mark expected values throughout.

2 Related Work

2.1 Sub-1B language models

The Pythia suite (Biderman et al., 2023) established the modern baseline for small-model pre-training, with Pythia-410M trained on 300B tokens of the Pile (Gao et al., 2020). Meta’s MobileLLM (Liu et al., 2024) demonstrated that at the sub-1B scale, *depth* is more valuable than *width* per parameter: a 350M-parameter model with 30 layers outperforms a comparably-sized wider model on reasoning benchmarks by substantial margins. HuggingFace’s SmolLM2 line (Allal et al., 2025) pushed the field forward by treating data quality (via FineWeb-Edu and Cosmopedia synthetic textbook data) and over-training (11T tokens on the 1.7B model) as the primary quality levers.

2.2 Code-specialized small models

DeepSeek-Coder (Guo et al., 2024) introduced repository-level sequence packing based on file-import topological sort, yielding substantial gains on cross-file code completion. The Qwen2.5-Coder family (Hui et al., 2024) demonstrated that a 494M-parameter model trained on 5.5T tokens of code-heavy data with fill-in-the-middle (FIM) formatting (Bavarian et al., 2022) can reach HumanEval pass@1 of 40.5, setting the current small-model state of the art.

2.3 Attention mechanism variants

Grouped-Query Attention (Ainslie et al., 2023) reduces the KV-cache memory footprint by sharing K and V across groups of Q heads, with negligible quality loss up to sharing ratios of 8:1. Mistral-7B (Jiang et al., 2023) introduced sliding-window attention, later combined with full attention in Gemma-2 (Team, 2024) to yield an alternating pattern that preserves long-range integration while reducing per-layer attention FLOPs. QK-normalization (Chameleon Team, 2024; Dehghani et al., 2023) applies RMSNorm to queries and keys prior to the dot product, preventing attention entropy collapse in late training.

Differential Attention (Ye et al., 2024), proposed by Microsoft Research in late 2024, computes two attention distributions with independently-projected query and key matrices and returns their weighted difference. The stated motivation is analogous to a differential amplifier in analog circuits: common-mode noise present in both attention distributions cancels, while informative signal that differs between the two branches survives. Ablations at 3B parameters show consistent perplexity improvements and reduced hallucination on question-answering

benchmarks. To our knowledge, no released model below 3B parameters currently uses this mechanism.

2.4 Training-recipe innovations

Multi-Token Prediction (Gloeckle et al., 2024) adds auxiliary heads that predict tokens $t + 2, t + 3, \dots$ at training time, discarded at inference. DeepSeek-V3 (DeepSeek-AI, 2024) demonstrates that MTP improves both convergence speed and downstream generation quality. Register tokens (Darcet et al., 2024), originally proposed for vision transformers, provide dedicated "attention sinks" that reduce spurious attention on semantically empty tokens. The Muon optimizer (Jordan, 2024), which applies orthogonal matrix updates via Newton-Schulz iteration, has demonstrated 1.5–2 $\times$ sample efficiency over AdamW on small-model benchmarks in the nanoGPT speedrun community.

3 void_v1 Architecture

3.1 Design principles

Three principles govern the design:

1. **Depth over width.** At $\leq 1\text{B}$ parameters, we allocate the parameter budget toward more layers rather than wider hidden dimensions, following the MobileLLM ablations.
2. **One controlled novelty.** Exactly one component (Differential Attention) is a novel bet; all other choices are drawn from published models with reproducible ablations. This keeps the debugging surface small when interpreting training failures.
3. **Graceful degradation.** Every novel or non-standard component has a documented roll-back path (§7) that preserves the ability to ship a reduced-scope model if the component underperforms.

3.2 Overall configuration

Let d denote the hidden dimension, L the number of transformer layers, H the number of query heads, H_{kv} the number of key/value heads (with $H \bmod H_{kv} = 0$), $d_h = d/H$ the head dimension, d_{ff} the feed-forward inner dimension, V the vocabulary size, and S the maximum sequence length. The full configuration is:

The total parameter count N derives from the configuration as follows. Let $T_{\text{embed}} = V \cdot d = 49,152 \cdot 960 = 4.72 \times 10^7$ (counted once due to embedding tying). Per transformer layer, the attention module contributes

$$\begin{aligned}
 T_{\text{attn}} &= \underbrace{2d \cdot Hd_h}_{2 \text{ Q projections}} + \underbrace{2d \cdot H_{kv}d_h}_{2 \text{ K projections}} + \underbrace{d \cdot H_{kv}d_h}_{1 \text{ V projection}} + \underbrace{Hd_h \cdot d}_{\text{output projection}} \\
 &+ \underbrace{d + 4d_h}_{\text{RMSNorm, QK-norm, } \lambda} \\
 &= 2 \cdot 960 \cdot 960 + 2 \cdot 960 \cdot 320 + 960 \cdot 320 + 960 \cdot 960 + (\text{negligible}) \\
 &\approx 3.17 \times 10^6.
 \end{aligned} \tag{1}$$

Note the "2 Q projections" and "2 K projections" terms: Differential Attention (§3.3) uses two independently-projected Q and K matrices. V is shared across both branches. This adds $d \cdot (Hd_h + H_{kv}d_h) = 960 \cdot (960 + 320) \approx 1.23 \times 10^6$ parameters per layer compared to a standard GQA attention module, an overhead of $\sim 63\%$ on the attention parameters and $\sim 8\%$ on the whole model (since attention is only part of each layer).

Hyperparameter	Value
Hidden dimension d	960
Number of transformer layers L	32
Query heads H	15
Key/Value heads H_{kv} (GQA ratio 3:1)	5
Head dimension d_h	64
FFN inner dimension d_{ff}	2560
Vocabulary size V	49,152
Max sequence length S	2048
RoPE frequency base θ	500,000
Register tokens R	4
Number of MTP heads (training only)	2 (predicting $t + 2, t + 3$)
Tied input/output embeddings	Yes
Normalisation	RMSNorm ($\epsilon = 10^{-5}$)
Activation (FFN)	SwiGLU
Attention pattern	See §3.4
Attention head normalisation	QK-norm on both branches
Bias in linear layers	None
Weight init	$\mathcal{N}(0, 0.02)$, residual projs $\times 1/\sqrt{2L}$
Dropout	0

Table 1: void_v1 architectural configuration.

The SwiGLU FFN contributes

$$T_{\text{ffn}} = 3 \cdot d \cdot d_{\text{ff}} = 3 \cdot 960 \cdot 2560 = 7.37 \times 10^6. \quad (2)$$

Total per layer: $T_{\text{attn}} + T_{\text{ffn}} + T_{\text{norms}} \approx 10.55 \times 10^6$.

The final RMSNorm adds $d = 960$ parameters. The MTP heads add $2 \cdot d \cdot V = 2 \cdot 960 \cdot 49,152 \approx 9.44 \times 10^7$ parameters, but these are (a) discarded at inference and (b) can be tied to the main LM head to eliminate the cost entirely; we adopt full tying in void_v1.

Summing:

$$N \approx T_{\text{embed}} + L \cdot T_{\text{layer}} + O(d) = 4.72 \times 10^7 + 32 \cdot 1.055 \times 10^7 \approx 4.85 \times 10^8. \quad (3)$$

i.e. $N \approx 485\text{M}$ parameters, consistent with the " $\sim 480\text{M}$ " claim in the abstract.

3.3 Differential Grouped-Query Attention

The core novel component. For an input sequence $\mathbf{X} \in \mathbb{R}^{S \times d}$, standard scaled dot-product attention with GQA computes

$$Q = \mathbf{X}W_Q, \quad W_Q \in \mathbb{R}^{d \times H d_h}, \quad (4)$$

$$K = \mathbf{X}W_K, \quad W_K \in \mathbb{R}^{d \times H_{kv} d_h}, \quad (5)$$

$$V = \mathbf{X}W_V, \quad W_V \in \mathbb{R}^{d \times H_{kv} d_h}, \quad (6)$$

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right)V, \quad (7)$$

where M is a causal mask and K, V are broadcast across the H/H_{kv} Q heads that share each KV head.

Differential GQA introduces a second Q and K projection:

$$Q_1 = \text{RMSNorm}_{\text{QK}}(\text{RoPE}(\mathbf{X}W_{Q_1})), \quad (8)$$

$$Q_2 = \text{RMSNorm}_{\text{QK}}(\text{RoPE}(\mathbf{X}W_{Q_2})), \quad (9)$$

$$K_1 = \text{RMSNorm}_{\text{QK}}(\text{RoPE}(\mathbf{X}W_{K_1})), \quad (10)$$

$$K_2 = \text{RMSNorm}_{\text{QK}}(\text{RoPE}(\mathbf{X}W_{K_2})), \quad (11)$$

$$V = \mathbf{X}W_V. \quad (12)$$

The attention scores are then combined:

$$\text{DiffAttn}(\mathbf{X}) = \left[\text{softmax}\left(\frac{Q_1K_1^\top}{\sqrt{d_h}} + M\right) - \lambda(t) \cdot \text{softmax}\left(\frac{Q_2K_2^\top}{\sqrt{d_h}} + M\right) \right] V, \quad (13)$$

where $\lambda(t) \in \mathbb{R}$ is a learnable, time-varying scalar per layer:

$$\lambda(t) = \exp(\lambda_{q_1} \cdot \lambda_{k_1}) - \exp(\lambda_{q_2} \cdot \lambda_{k_2}) + \lambda_{\text{init}}, \quad (14)$$

with $\lambda_{q_i}, \lambda_{k_i} \in \mathbb{R}^{d_h}$ learnable and $\lambda_{\text{init}} = 0.8 - 0.6 \exp(-0.3 \cdot (\ell - 1))$ for layer index ℓ , following the schedule of [Ye et al. \(2024\)](#). This schedule ensures $\lambda(0) \approx 0.2$ in the deepest layer and $\lambda(0) \approx 0.8$ in shallow layers at initialization, biasing early layers toward stronger differential subtraction.

Cost. Compared to standard GQA, differential GQA doubles the Q and K projection parameters (V unchanged) and roughly doubles the attention score computation (two softmaxes). For a training-compute breakdown, see §4.5.

3.4 Mixed attention pattern

Not all layers use full attention. Following the Mistral/Gemma-2 design, we adopt a mixed pattern:

- **Layers 1–4:** full attention (early integration of broad context).
- **Layers 5–24:** alternating block of {sliding-window (window=512), sliding-window, sliding-window, full}, repeated 5 times.
- **Layers 25–32:** full attention (final high-level integration).

This distributes 12 full-attention and 20 sliding-window layers, reducing per-token attention FLOPs by approximately 40% at sequence length 2048 while preserving the ability of the network to integrate long-range context.

3.5 Multi-Token Prediction (MTP) heads

At training time only, two auxiliary linear heads ϕ_2, ϕ_3 take the final hidden state h_t and predict tokens at positions $t + 2$ and $t + 3$ respectively. The training loss is

$$\mathcal{L}_{\text{MTP}} = \mathcal{L}_{\text{CE}}(h_t, x_{t+1}) + \alpha_2 \mathcal{L}_{\text{CE}}(\phi_2(h_t), x_{t+2}) + \alpha_3 \mathcal{L}_{\text{CE}}(\phi_3(h_t), x_{t+3}), \quad (15)$$

with $\alpha_2 = \alpha_3 = 0.3$ following the DeepSeek-V3 recipe. The MTP heads share weights with the main LM head via linear projections of the hidden state through separate small transformer blocks (one layer each), an approach that adds $\sim 2\text{M}$ parameters per MTP head and is discarded at inference.

3.6 Register tokens

Four learnable register tokens $\mathbf{r}_1, \dots, \mathbf{r}_4 \in \mathbb{R}^d$ are prepended to every input sequence, making the effective input $[\mathbf{r}_1, \dots, \mathbf{r}_4, x_1, \dots, x_S]$. These tokens receive no supervision signal (their positions are masked in the loss) and serve as dedicated attention sinks, reducing attention entropy on the semantically meaningful tokens ([Darcet et al., 2024](#)).

3.7 Optimizer choice

We use **Muon** (Jordan, 2024) for hidden-layer weight matrices (attention Q/K/V/O projections, FFN gate/up/down) and **AdamW** (Loshchilov & Hutter, 2019) for embedding, LM head, norms, λ -scalars, and biases (of which there are none in linear layers by design). Muon applies an orthogonal step:

$$W_{t+1} = W_t - \eta \cdot \text{NewtonSchulz}_5(G_t), \quad (16)$$

where G_t is the gradient and NewtonSchulz_5 approximates the polar-decomposition orthogonal factor via five Newton-Schulz iterations. Empirically, Muon achieves 1.5–2 $\times$ sample efficiency over AdamW on nanoGPT-scale benchmarks and is our primary lever for training-token efficiency.

4 Theoretical Analysis

We now formalise three claims that motivate the `void_v1` architecture.

4.1 Attention noise decomposition

Definition 1 (Attention noise). Fix a query index i and a key sequence of length n . Let $A_i = \text{softmax}(q_i K^\top / \sqrt{d_h})$ be the row- i attention distribution. Let $\mathcal{S} \subseteq \{1, \dots, n\}$ be the set of *semantically relevant* key positions for query i (under some oracle relevance measure). Define the **off-diagonal mass** as

$$\mathcal{N}(A_i) = \sum_{j \notin \mathcal{S}} A_{i,j}. \quad (17)$$

$\mathcal{N}(A_i) \in [0, 1]$ measures the fraction of attention "leaked" onto irrelevant tokens.

Assumption 1 (Common-mode noise). For any query i , there exist *signal* distributions $A_1^{(s)}, A_2^{(s)} \in \mathbb{R}^n$ (concentrated on $\mathcal{S}$) and a *shared noise* distribution $\mathcal{N}_c \in \mathbb{R}^n$ (concentrated on $\{1, \dots, n\} \setminus \mathcal{S}$) such that the standard-attention outputs of the two branches decompose as

$$A_i^{(1)} = A_1^{(s)} + \mathcal{N}_c + \epsilon_1, \quad (18)$$

$$A_i^{(2)} = A_2^{(s)} + \mathcal{N}_c + \epsilon_2, \quad (19)$$

where ϵ_1, ϵ_2 are independent zero-mean residuals with $\mathbb{E}[\epsilon_i] = 0$ and $\text{Var}(\epsilon_i) = \sigma^2$.

Assumption 1 is the key modeling step. It asserts that two attention branches operating on the same input, but with different Q/K projections, will produce (i) different attempts at capturing the true signal, and (ii) similar noise patterns on the irrelevant-token subset $\{1, \dots, n\} \setminus \mathcal{S}$, because that noise arises from common input features (e.g. high-frequency positional artifacts, repeated sub-tokens, embedding-space clustering).

4.2 Differential attention noise reduction

Theorem 1 (Off-diagonal mass reduction). Let $A_i^{\text{DiffAttn}} = A_i^{(1)} - \lambda A_i^{(2)}$ denote the (unrenormalised) differential attention row for query i , with $\lambda \in [0, 1]$. Under Assumption 1,

$$\mathbb{E}[\mathcal{N}(A_i^{\text{DiffAttn}})] = (1 - \lambda) \mathbb{E}[\mathcal{N}(\mathcal{N}_c)] + \mathbb{E}[\mathcal{N}(A_1^{(s)} - \lambda A_2^{(s)})] + O(\sigma). \quad (20)$$

In particular, for $\lambda = 1$, the shared-noise contribution vanishes:

$$\mathbb{E}[\mathcal{N}(A_i^{\text{DiffAttn}})]_{\lambda=1} = \mathbb{E}[\mathcal{N}(A_1^{(s)} - A_2^{(s)})] + O(\sigma). \quad (21)$$

Proof sketch. By linearity of expectation and Assumption 1,

$$\mathcal{N}(A_i^{\text{DiffAttn}}) = \sum_{j \notin \mathcal{S}} (A_{i,j}^{(1)} - \lambda A_{i,j}^{(2)}) \quad (22)$$

$$= \sum_{j \notin \mathcal{S}} \left[(A_{1,j}^{(s)} - \lambda A_{2,j}^{(s)}) + (1 - \lambda) \mathcal{N}_{c,j} + (\epsilon_{1,j} - \lambda \epsilon_{2,j}) \right]. \quad (23)$$

Since $A_1^{(s)}, A_2^{(s)}$ are concentrated on $\mathcal{S}$, the first term contributes to $\mathcal{N}(\cdot)$ only through residual leakage, which we absorb into $\mathbb{E}[\mathcal{N}(A_1^{(s)} - \lambda A_2^{(s)})]$. Taking expectations, the residual term $\mathbb{E}[\epsilon_{1,j} - \lambda \epsilon_{2,j}] = 0$ contributes $O(\sigma)$ (with the constant depending on $|\mathcal{S}^c|$ and λ). The shared-noise term scales linearly with $(1 - \lambda)$, giving the claimed decomposition. $\square$

Remark 1. The theorem does *not* claim that A_i^{DiffAttn} is itself a probability distribution — it is not, in general. Both branches independently produce valid softmax outputs; their difference lies in $[-1, 1]$ pointwise. In practice, the raw difference is then multiplied by V and fed through subsequent RMSNorm layers, so the value-vector mixing recovers well-scaled activations. This is the standard treatment in Ye et al. (2024). An alternative interpretation is that DiffAttn produces a signed "attention field" rather than a probability, and λ acts as a gain control on the second branch.

Corollary 1 (Practical implication). *When Assumption 1 holds strongly (high common-mode noise, low residual variance), Differential Attention with $\lambda \rightarrow 1$ produces cleaner attention over informative tokens. When $\lambda \rightarrow 0$, differential attention reduces to standard attention on the first branch; the second branch is then a redundant regularizer. The learnable schedule for $\lambda(t)$ (§3.3) allows the model to interpolate between these regimes per layer and per training step.*

4.3 Parameter efficiency of GQA

Proposition 1 (GQA parameter savings). *Let $\rho = H/H_{kv}$ denote the GQA sharing ratio. Compared to multi-head attention ($\rho = 1$), grouped-query attention saves*

$$\Delta T_{\text{attn}} = 2 \cdot d \cdot d_h \cdot H \cdot \left(1 - \frac{1}{\rho}\right) \quad (24)$$

parameters per layer on the K and V projections combined. For void_v1 ($d = 960$, $d_h = 64$, $H = 15$, $\rho = 3$), this saves $2 \cdot 960 \cdot 64 \cdot 15 \cdot (2/3) \approx 1.23 \times 10^6$ parameters per layer, or $\sim 39M$ parameters across the 32-layer model — a $\sim 8\%$ reduction in total parameter count for negligible quality loss per published GQA ablations (Ainslie et al., 2023). Moreover, the KV cache at inference shrinks by $1 - 1/\rho = 2/3$, reducing memory pressure during generation.

4.4 MTP as implicit regularization

Proposition 2 (MTP as multi-task regularization). *The MTP training objective*

$$\mathcal{L}_{\text{MTP}} = \mathcal{L}_{\text{CE}}(h_t, x_{t+1}) + \sum_{k=2}^K \alpha_k \mathcal{L}_{\text{CE}}(\phi_k(h_t), x_{t+k}) \quad (25)$$

can be viewed as a multi-task learning objective in which the shared representation h_t must simultaneously support prediction of the immediate next token and of tokens further into the future. By the standard multi-task learning analysis (Caruana, 1997), this biases h_t toward representations that capture longer-range predictive structure rather than only local next-token statistics, providing an implicit regularization against overfitting to short-range patterns.

This is a heuristic argument, not a rigorous bound. The empirical evidence from DeepSeek-V3 (which reports both faster convergence and better downstream evals with MTP heads) supports the interpretation.

4.5 FLOPs and memory budget

Forward-pass FLOPs. For one token at sequence position t with sequence length S :

- Attention Q/K/V projections: $2 \cdot d \cdot (2Hd_h + 2H_{kv}d_h + H_{kv}d_h) \approx 6.34 \times 10^6$ FLOPs (dominated by diff-attn’s $2 \times$ Q/K).
- Attention scores: $2 \cdot S \cdot Hd_h \cdot 2 \approx 4S \cdot d$ FLOPs (two softmaxes over S).
- Attention output projection: $2 \cdot d \cdot d \approx 1.84 \times 10^6$ FLOPs.
- FFN: $2 \cdot 3 \cdot d \cdot d_{ff} = 1.47 \times 10^7$ FLOPs.
- Per-layer total: $\approx 2.3 \times 10^7$ FLOPs per token at $S = 2048$.
- 32 layers: $\approx 7.4 \times 10^8$ FLOPs per token.
- Embedding + final head (tied): $2 \cdot d \cdot V \approx 9.4 \times 10^7$ FLOPs per token.
- MTP heads (train only): additional $\sim 3 \times 10^7$ FLOPs.

Total forward: $\approx 8.4 \times 10^8$ FLOPs/token (train), or $\approx 8.1 \times 10^8$ FLOPs/token (inference, no MTP).

Backward pass. Approximately $2 \times$ the forward pass. Total training FLOPs per token: $\approx 2.5 \times 10^9$.

Training-run FLOP budget. For 100B tokens: $100 \times 10^9 \cdot 2.5 \times 10^9 = 2.5 \times 10^{20}$ FLOPs. On a Kaggle TPU v3-8 at ~ 180 TFLOPS sustained, this is ~ 386 compute-hours, or approximately 4–5 months of wall-clock at 20 hours/week free allocation. On a rented H100 at ~ 400 TFLOPS sustained, this is ~ 175 hours, or one training run for roughly \$350–700 at spot pricing.

Memory (bf16 mixed precision, single GPU). Model parameters (bf16): $2 \cdot 485 \times 10^6 = 0.97$ GB. Gradients (bf16): 0.97 GB. Optimizer states (fp32 for AdamW parameters, Muon has smaller state): ~ 3.5 GB. Activations at $S = 2048$, batch = 8, with gradient checkpointing at every 4 layers: ~ 4 GB. KV cache during generation (GQA, batch = 1, full 2048 context): $2 \cdot L \cdot H_{kv} \cdot d_h \cdot S \cdot 2 = 2 \cdot 32 \cdot 5 \cdot 64 \cdot 2048 \cdot 2 = 168$ MB. **Total peak training memory:** ~ 10 GB — fits comfortably on a 24 GB consumer GPU or on Kaggle T4 $\times 2$ (with FSDP or DDP).

5 Training Methodology

5.1 Data pipeline

The pretraining corpus comprises 100B tokens drawn from six sources with the following mixture weights:

Source	Share	Filter
The Stack v2 dedup (Python, JS, TS, Rust, Go, C, C++, Java)	55%	stars ≥ 5 , permissive licenses
FineWeb-Edu	20%	educational score ≥ 3
Cosmopedia v2 (synthetic textbooks)	10%	—
OpenWebMath + AutoMathText	5%	math reasoning
GitHub issues + PR discussions	5%	code-adjacent NL
StackExchange (code tags) + Wikipedia (en)	5%	Q&A, general knowledge floor

Table 2: Pretraining data mixture (100B tokens total).

The pipeline applies (in order): language and quality filtering, MinHash deduplication, contamination scrubbing against every planned evaluation benchmark (HumanEval, MBPP, MMLU, GSM8K, ARC, HellaSwag, plus our novel void-cli-bench), FIM reformatting on 50% of code samples, and repository-aware sequence packing for the Stack subset. Tokenization uses a BPE tokenizer with 49,152 tokens, including FIM sentinels and ChatML special tokens for later SFT compatibility.

5.2 Curriculum: three-stage annealing

Following Allal et al. (2025), we adopt a three-stage curriculum:

- **Stage 1 (85B tokens):** Broad mix per Table 2, cosine learning rate schedule with peak 3×10^{-4} , linear warmup over 2000 steps.
- **Stage 2 (10B tokens):** Anneal data mixture toward code and math (Stack 60%, Cosmopedia 20%, OpenWebMath 15%, rest 5%). Learning rate decays via cosine to 3×10^{-5} .
- **Stage 3 (5B tokens):** Cosmopedia and highest-quality code only. Add teacher distillation objective (§5.3). Learning rate anneals to near zero.

This three-stage schedule is empirically worth 2–5 downstream evaluation points at negligible compute cost.

5.3 Knowledge distillation

During Stage 3, we combine the standard cross-entropy loss with a Kullback-Leibler divergence to the output distribution of Qwen2.5-Coder-7B (Hui et al., 2024) (chosen as a code-specialist teacher rather than a general-purpose teacher):

$$\mathcal{L}_{\text{stage3}} = (1 - \beta) \mathcal{L}_{\text{CE}}(p_{\text{student}}, y) + \beta \cdot T^2 \cdot \text{KL}(p_{\text{teacher}}^T \| p_{\text{student}}^T), \quad (26)$$

with temperature $T = 2$ and distillation weight $\beta = 0.5$. Teacher logits are computed once and cached on HuggingFace Hub as compressed tensors (top- $k = 100$ per position, saving $\sim 400\times$ storage over full logits) prior to Stage 3 training.

5.4 Post-training

We produce two release variants:

- **void-v1-base:** the pretrained model, model-souped from the last three Stage 3 checkpoints via uniform averaging.
- **void-v1-instruct:** base + supervised fine-tuning on 100k instruction samples (self-oss-instruct-sc2-exec-filter-50k, OpenHermes-2.5 subset, no_robots) + Direct Preference Optimization (Rafailov et al., 2023) on 30k pairs of UltraFeedback-clean.

6 Theoretical Comparison with Reference Models

Table 3 places void_v1 alongside five reference models in the 350M–1.3B parameter range. All quality figures for reference models are published; the void_v1 column shows the configuration targets and *expected* evaluation numbers under the training recipe of §5.

Feature	Pythia-410M	MobileLLM-350M	SmolLM2-360M	Qwen2.5-C-0.5B	DeepSeek-C-1.3B	
Params (M)	410	350	362	494	1300	
Layers	24	30	32	24	24	
Hidden	1024	960	960	896	2048	
GQA ratio	1 (MHA)	3	3	7	1 (MHA)	
RoPE θ	10k	10k	10k	1M	10k	
Attention pattern	full	full	full	full	full	
QK-norm	no	no	no	no	no	
Differential attention	no	no	no	no	no	
MTP heads	no	no	no	no	no	
Register tokens	no	no	no	no	no	
FIM	no	no	no	yes	yes	
Optimizer	AdamW	AdamW	AdamW	AdamW	AdamW	
Training tokens	300B	1T	4T	5.5T	2T	
Distillation	no	no	no	no	no	stage
HumanEval (pass@1)	2.4	8.6	12.8	40.5	34.8	
MBPP (pass@1)	~ 3	—	15.6	40.4	55.6	
MMLU	25.7	25.7	24.7	30.5	—	
HellaSwag	40.6	49.6	54.6	55.5	—	

Table 3: Theoretical comparison of void_v1 with reference sub-1.3B models. Reference numbers from published model cards and technical reports. void_v1 numbers are *expected values* conditioned on the training recipe of §5; see §6.1 for the derivation.

6.1 How expected values are derived

The expected value ranges in Table 3 are derived as follows:

1. **Anchor point.** Qwen2.5-Coder-0.5B (494M parameters, 5.5T tokens) establishes the *ceiling* for the size class on HumanEval and MBPP at ~ 40 . This is a $55\times$ over-training margin relative to Chinchilla-optimal for 494M.
2. **Under-training penalty.** void_v1 targets 100B tokens ($\sim 20\times$ Chinchilla-optimal for 485M parameters). By the Chinchilla scaling laws (Hoffmann et al., 2022), halving the training-token budget of a small model incurs a ~ 0.02 loss increase in cross-entropy, which corresponds empirically to $\sim 20\text{--}30\%$ HumanEval degradation. Extrapolating from Qwen’s 40.5 to our $55\times$ smaller token budget gives a naïve floor of $\sim 20\text{--}25$.
3. **Data-quality bonus.** FineWeb-Edu score-filtered data and Cosmopedia synthetic textbooks demonstrate $\sim 5\text{--}8$ -point improvements on HellaSwag/MMLU for comparable models (Allal et al., 2025), and analogous gains are expected for code metrics from our filtered Stack v2 subset.
4. **Distillation bonus.** Stage-3 distillation from Qwen2.5-Coder-7B is expected to add $\sim 3\text{--}6$ points on HumanEval based on published distillation work (Team, 2024).
5. **Differential Attention bonus (speculative).** The published DIFF-Transformer ablations at 3B parameters show a $\sim 5\text{--}10\%$ perplexity improvement, which historically translates to $\sim 1\text{--}3$ downstream evaluation points at our scale. This is our research bet: the value could plausibly be 0 (no help at this scale) or as high as ~ 5 points (matching published ablations at larger scale).

Summing the anchor floor with these adjustments yields the expected HumanEval range of $\sim 25\text{--}32$. We stress this is a design-time projection; the actual outcome will be determined empirically.

7 Risks and Ablation Plan

Every novel or non-standard component carries a documented rollback path:

Component	Failure mode	Rollback
Differential Attention	Training diverges or degrades quality after 20B tokens	Set $W_{Q_2} = W_{K_2} = 0$ at init, $\lambda = 0$; falls back to standard GQA.
Multi-Token Prediction	Auxiliary loss dominates or destabilises training	Set $\alpha_2 = \alpha_3 = 0$; heads are discarded and only next-token loss remains.
Register tokens	No measurable benefit at 20B-token milestone	Remove; retraining continues from the checkpoint with 4 fewer positions per sequence.
Muon optimizer	Instability or NaN loss	Swap to pure AdamW at any checkpoint; parameter update rule change only.
Mixed attention pattern	Long-range integration failure on evals	Convert all sliding-window layers to full attention; adds $\sim 40\%$ FLOPs per layer.
Teacher distillation (Stage 3)	Teacher too different from student $\rightarrow$ destabilises anneal	Drop the KL term; Stage 3 reduces to standard cross-entropy on Cosmopedia.

Table 4: Risk register with graceful-degradation rollbacks.

Fail-fast evaluation gates. We commit to evaluate at 20B, 50B, 85B, 95B, and 100B tokens. If HumanEval at the 20B-token milestone is below 8 (the SmolLM2-360M level with $50\times$ our tokens), we halt training, diagnose, and consider rollback before spending the remaining 80% of compute budget.

8 Evaluation

8.1 Standard benchmarks

We report on `bigcode-evaluation-harness` (HumanEval, HumanEval-FIM, MBPP) and `lm-eval-harness` (MMLU, HellaSwag, ARC-Easy, ARC-Challenge, PIQA, WinoGrande, GSM8K with chain-of-thought). All benchmarks are contamination-checked against the training corpus.

8.2 void-cli-bench

We introduce a novel domain-specific evaluation: a suite of ~ 200 command-line reasoning tasks drawn from open-source CLI codebases (fish, zsh, ripgrep, fzf, bat, exa) covering (a) command completion given partial input and history, (b) shell-script bug localisation, (c) man-page-based argument recall, and (d) pipeline construction from natural-language specifications. Contamination checks against the CLI codebase source are enforced at data-pipeline time.

9 Conclusion

We have specified the design of `void_v1`, a 485M-parameter decoder-only language model that combines a proven-in-2024 core (deep-narrow scaling, GQA, RoPE, RMSNorm, SwiGLU, mixed attention, MTP, register tokens, QK-norm, Muon optimizer) with one deliberately novel bet: Differential Attention at the sub-1B scale. We formalised the noise-cancellation argument for Differential Attention as a theorem on expected off-diagonal attention mass, derived the parameter and FLOP budgets from first principles, and placed the design in a theoretical comparison with five reference models. The design paper commits us to a specific training

recipe (100B tokens across three curriculum stages, teacher distillation from Qwen2.5-Coder-7B, Muon+AdamW hybrid optimization) and to a fail-fast evaluation protocol with documented rollbacks for every novel component.

The empirical validation of every claim in this document remains open. The next milestone is a 10M-parameter debug configuration trained on ~ 100 M tokens, which will validate the pipeline and provide the first empirical read on whether Differential Attention produces cleaner attention maps at our scale before we commit to the full run.

References

- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., & Sanghai, S. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv:2305.13245*.
- Allal, L. B., et al. (2025). SmolLM2: When Smol Goes Big — Data-Centric Training of a Small Language Model. *arXiv:2502.02737*.
- Bavarian, M., Jun, H., Tezak, N., et al. (2022). Efficient Training of Language Models to Fill in the Middle. *arXiv:2207.14255*.
- Biderman, S., et al. (2023). Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. *ICML*.
- Caruana, R. (1997). Multitask Learning. *Machine Learning*, 28, 41–75.
- Chameleon Team. (2024). Chameleon: Mixed-Modal Early-Fusion Foundation Models. *arXiv:2405.09818*.
- Darcet, T., Oquab, M., Mairal, J., & Bojanowski, P. (2024). Vision Transformers Need Registers. *ICLR*.
- DeepSeek-AI. (2024). DeepSeek-V3 Technical Report. *arXiv:2412.19437*.
- Dehghani, M., et al. (2023). Scaling Vision Transformers to 22 Billion Parameters. *ICML*.
- Gao, L., et al. (2020). The Pile: An 800GB Dataset of Diverse Text for Language Modeling. *arXiv:2101.00027*.
- Gloeckle, F., Idrissi, B. Y., Rozière, B., et al. (2024). Better & Faster Large Language Models via Multi-token Prediction. *arXiv:2404.19737*.
- Guo, D., et al. (2024). DeepSeek-Coder: When the Large Language Model Meets Programming — The Rise of Code Intelligence. *arXiv:2401.14196*.
- Hoffmann, J., et al. (2022). Training Compute-Optimal Large Language Models. *arXiv:2203.15556*.
- Hui, B., et al. (2024). Qwen2.5-Coder Technical Report. *arXiv:2409.12186*.
- Jiang, A. Q., et al. (2023). Mistral 7B. *arXiv:2310.06825*.
- Jordan, K. (2024). Muon: An Optimizer for Hidden Layers in Neural Networks. <https://kellerjordan.github.io/posts/muon/>.
- Liu, Z., Zhao, C., Iandola, F., et al. (2024). MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. *arXiv:2402.14905*.
- Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization. *ICLR*.
- Rafailov, R., et al. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. *NeurIPS*.
- Shazeer, N. (2020). GLU Variants Improve Transformer. *arXiv:2002.05202*.

- Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*, 568, 127063.
- Gemma Team. (2024). Gemma 2: Improving Open Language Models at a Practical Size. *arXiv:2408.00118*.
- Touvron, H., et al. (2023). LLaMA: Open and Efficient Foundation Language Models. *arXiv:2302.13971*.
- Ye, T., Dong, L., Xia, Y., et al. (2024). Differential Transformer. *arXiv:2410.05258*.
- Zhang, B., & Sennrich, R. (2019). Root Mean Square Layer Normalization. *NeurIPS*.